

ISL - TP 1

Detector de Números Primos de 4 bits

Felipe Pires Franco

23 de maio de 2026

Introdução

Um circuito identificador de primos de 4 bits deve ser capaz de receber 4 bits que formam um número e retornar 1 caso seja primo (2, 3, 5, 7, 11, 13), ou 0 caso não seja primo (números restantes de 0 até 15). Como o problema apresentado é simples e não precisa gerenciar memória - não armazena nenhum valor nem depende de valores armazenados, apenas dos inputs recebidos -, é possível solucionar o problema ao implementar um circuito combinacional, a partir do uso de somente portas lógicas simples.

Logo, para seguir com o desenvolvimento, seguiram-se os seguintes passos:

- Análise da tabela verdade, para obter a forma canônica;
- Simplificação da fórmula booleana, para facilitar a implementação
- Desenho do circuito;
- Desenvolvimento do código em verilog baseado no diagrama do circuito.

Essas são as etapas que serão analisadas ao longo desse documento.

1 Tabela Verdade

Para um número binário b , cada algarismo que compõe o número receberá o nome de variável de b_x , sendo x a potência de 2 que representa o valor do algarismo. Assim, o número de 4 bits analisado para o problema será composto de 4 variáveis: b_0 , b_1 , b_2 e b_3 , em que b_0 é o bit menos significativo e b_3 , o mais significativo. Além disso, o resultado do circuito receberá o nome de F .

A partir disso, podemos analisar a tabela verdade (ver Tabela 1)

Tabela 1: tabela verdade da função identificadora de primos

b_3	b_2	b_1	b_0	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0

Forma Canônica

A partir da tabela verdade, é possível construir a forma canônica ao pegar as linhas que resultando em verdade ($F=1$) para formar os mintermos - disjunção das variáveis normais (valoração 1) ou negadas (valoração 0) - e fazer a conjunção deles. Assim, obtemos:

$$F = \overline{b_3} \overline{b_2} b_1 \overline{b_0} + \overline{b_3} \overline{b_2} b_1 b_0 + \overline{b_3} b_2 \overline{b_1} b_0 + \overline{b_3} b_2 b_1 b_0 + b_3 \overline{b_2} b_1 b_0 + b_3 b_2 \overline{b_1} b_0$$

que pode ser escrito como:

$$F = \sum m(m2, m3, m5, m7, m11, m13)$$

2 Simplificação

A partir da forma canônica de mintermos, é ideal fazer a simplificação para facilitar a implementação. Para isso, como o problema utiliza de apenas 4 variáveis, é prático utilizar o mapa de Karnaugh como ferramenta de simplificação. Assim, o mapa de karnaugh é preenchido e os valores comuns são agrupados (ver figura 1).

A fim de fazer a forma simplificada de produto de somas (PoS), serão agrupados os zeros, em que as variáveis são normais para valoração 0 e negadas para valoração 1. Assim, forma-se a forma PoS a partir do mapa de Karnaugh (ver equação 1)

Figura 1: Mapa de karnough para os primos de 4 bits

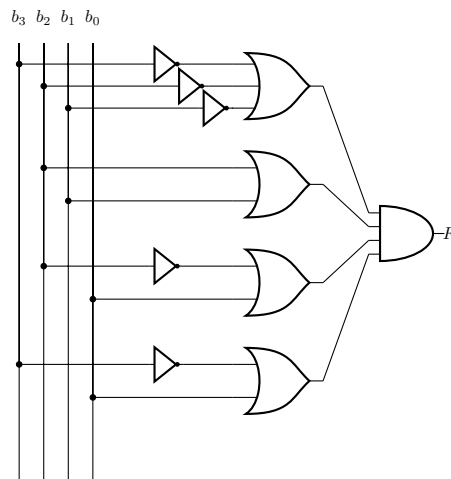
		b_1b_0			
		00	01	11	10
b_3b_2	00	0	0	1	1
	01	0	1	1	0
	11	0	1	0	0
	10	0	0	1	0

$$F = (\overline{b_3} + \overline{b_2} + \overline{b_1})(b_2 + b_1)(\overline{b_2} + b_0)(\overline{b_3} + b_0) \quad (1)$$

3 Circuito

Então, a partir da forma simplificada PoS, é possível implementar um circuito utilizando as variáveis b_0, b_1, b_2, b_3 para gerar F (resultado primo, 1, ou não primo, 0). Para isso, utiliza-se de portas OR para formar cada literal (ou sua negação) composto das somas booleanas e, depois, uni-los com uma porta AND, implementando toda a equação de F .

Com isso, temos o diagrama do circuito:



4 Verilog

Com o circuito planejado, é possível utilizar do verilog para implementar a lógica dele. O código da implementação e do testbench estão incluídos no zip. Já a partir do output e do diagrama de forma de onda do testbench (obtidos por meio da plataforma EDA Playground), é possível perceber seus resultado:

Listing 1: Resultado da simulação no terminal do EDA Playground.

VCD info: dumpfile dump_prime.vcd opened for output.						
Num	B3	B2	B1	B0	Saida F	
----	----	----	----	----	-----	----
0	0	0	0	0	0	
1	0	0	0	1	0	
2	0	0	1	0	1	
3	0	0	1	1	1	
4	0	1	0	0	0	
5	0	1	0	1	1	
6	0	1	1	0	0	
7	0	1	1	1	1	
8	1	0	0	0	0	
9	1	0	0	1	0	
10	1	0	1	0	0	
11	1	0	1	1	1	
12	1	1	0	0	0	
13	1	1	0	1	1	
14	1	1	1	0	0	
15	1	1	1	1	0	

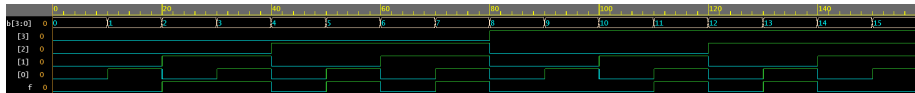


Figura 2: Forma de ondas do código verilog